Papan Tulis Virtual dengan Deteksi Tangan

Kode ini mengimplementasikan papan tulis virtual sederhana yang memungkinkan pengguna untuk menggambar, menghapus, dan mengganti warna menggunakan gerakan tangan yang dideteksi oleh MediaPipe Hand Landmarker.

1. Persiapan Lingkungan dan Import Library

Bagian ini mengimpor semua library yang diperlukan dan mengatur konfigurasi awal untuk aplikasi.

```
import cv2
import mediapipe as mp
import numpy as np
import time
from mediapipe.tasks.python import vision
from mediapipe.tasks.python.core.base_options import BaseOptions

# ================= CONFIG =================
MODEL_PATH = r"C:\codingan\project\hand_landmarker.task" # Jalur ke model MediaPipe Hand Landmarker
SMOOTHING_ALPHA = 0.35 # Faktor smoothing untuk pergerakan jari
BRUSH_SIZE = 8 # Ukuran kuas untuk menggambar
GESTURE_CONFIRM_FRAMES = 4 # Jumlah frame berturut-turut untuk mengonfirmasi gestur

# Warna default kuas (BGR)
current_color = (255, 0, 0) # Biru

# Palet warna yang tersedia (BGR)
color_palette = [
    (255, 0, 0),     # Biru
    (0, 255, 0),     # Hijau
    (0, 0, 255),     # Merah
    (0, 255, 255),   # Kuning
    (255, 0, 255),   # Ungu
    (0, 165, 255)    # Oranye
]
```

Penjelasan Variabel Konfigurasi:

- `MODEL_PATH`: Menentukan lokasi file model `hand_landmarker.task` yang digunakan oleh MediaPipe.
- `SMOOTHING_ALPHA`: Nilai antara 0 dan 1. Mengontrol seberapa banyak posisi jari saat ini memengaruhi posisi `smoothed_point`. Nilai yang lebih tinggi berarti lebih sedikit smoothing, nilai yang lebih rendah berarti lebih banyak smoothing.
- `BRUSH_SIZE`: Menentukan ketebalan garis saat menggambar.
- `GESTURE_CONFIRM_FRAMES`: Jumlah frame berturut-turut di mana suatu gestur harus terdeteksi agar dianggap sebagai gestur yang valid. Ini membantu mencegah deteksi gestur yang tidak disengaja atau goyangan kecil.
- `current_color`: Warna kuas yang sedang aktif, diatur sebagai tuple BGR.
- `color_palette`: Daftar warna yang dapat dipilih oleh pengguna, juga dalam format BGR.

2. Inisialisasi MediaPipe Hand Landmarker

Bagian ini mengatur dan menginisialisasi model deteksi tangan MediaPipe.

```
# ================= MEDIAPIPE INIT =================
options = vision.HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=MODEL_PATH),
    running_mode=vision.RunningMode.VIDEO,
    num_hands=1
)

landmarker = vision.HandLandmarker.create_from_options(options)
```

Penjelasan:

- `vision.HandLandmarkerOptions`: Digunakan untuk mengonfigurasi `HandLandmarker`.
  - `base_options=BaseOptions(model_asset_path=MODEL_PATH)`: Menunjuk ke jalur model yang telah ditentukan sebelumnya.
  - `running_mode=vision.RunningMode.VIDEO`: Menunjukkan bahwa model akan memproses stream video, bukan gambar statis.
  - `num_hands=1`: Mengonfigurasi model untuk mendeteksi maksimal satu tangan.
- `landmarker = vision.HandLandmarker.create_from_options(options)`: Membuat instance detektor tangan menggunakan opsi yang telah ditentukan.

3. Pengaturan Kamera dan Variabel Aplikasi

Bagian ini menyiapkan akses kamera dan variabel-variabel awal yang dibutuhkan untuk logika papan tulis.

```
# ================= CAMERA =================
cap = cv2.VideoCapture(0) # Menginisialisasi kamera default (indeks 0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 960) # Mengatur lebar frame kamera
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720) # Mengatur tinggi frame kamera

canvas = None # Layer tempat gambar akan dibuat
prev_point = None # Titik sebelumnya untuk menggambar garis
smoothed_point = None # Titik jari yang di-smoothing

draw_counter = 0 # Konter untuk gestur menggambar
erase_counter = 0 # Konter untuk gestur menghapus
fist_counter = 0 # Konter untuk gestur kepalan tangan (pilih warna)
gesture_state = "NONE" # Status gestur saat ini
```

Penjelasan:

- `cap = cv2.VideoCapture(0)`: Membuka stream video dari kamera web default.
- `cap.set(...)`: Mengatur resolusi frame yang diambil dari kamera.
- `canvas`: Sebuah array NumPy yang akan berfungsi sebagai "papan tulis" virtual. Awalnya `None` dan akan diinisialisasi dengan ukuran yang sama dengan frame kamera setelah frame pertama tersedia.
- `prev_point`: Menyimpan koordinat titik jari terakhir dari frame sebelumnya. Digunakan untuk menggambar garis kontinu.
- `smoothed_point`: Versi `index_point` yang telah di-smoothing untuk mengurangi jitter.
- `draw_counter`, `erase_counter`, `fist_counter`: Variabel ini melacak berapa banyak frame berturut-turut suatu gestur terdeteksi. Ini membantu mengimplementasikan `GESTURE_CONFIRM_FRAMES`.
- `gesture_state`: Menyimpan gestur yang sedang aktif (DRAW, ERASE, FIST, atau NONE).

4. Loop Utama Aplikasi

Ini adalah inti dari aplikasi, tempat frame video diproses, deteksi tangan dilakukan, gestur dikenali, dan tindakan menggambar/menghapus/memilih warna diimplementasikan.

```
print("Press Q to exit")

while True:
    ret, frame = cap.read() # Membaca frame dari kamera
    if not ret:
        break

    frame = cv2.flip(frame, 1) # Membalik frame secara horizontal (mirip cermin)
    h, w, _ = frame.shape # Mendapatkan tinggi, lebar, dan saluran warna frame

    if canvas is None:
        canvas = np.zeros_like(frame) # Inisialisasi kanvas kosong jika belum ada

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Mengonversi frame dari BGR ke RGB (untuk MediaPipe)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb) # Membuat objek MediaPipe Image
```

```
        timestamp = int(time.time() * 1000) # Mendapatkan timestamp saat ini dalam milidetik
        result = landmarker.detect_for_video(mp_image, timestamp) # Mendeteksi landmark tangan

        mode = "NONE" # Reset mode tampilan

        if result.hand_landmarks:
            lm = result.hand_landmarks[0] # Mengambil landmark tangan pertama yang terdeteksi

            def px(p):
                return int(p.x * w), int(p.y * h) # Fungsi bantu untuk mengonversi koordinat relatif ke piksel

            # ================= FINGER STATES =================
            # Mendeteksi apakah jari tertentu terangkat berdasarkan posisi y landmark
            index_up = lm[8].y < lm[6].y # Jari telunjuk
            middle_up = lm[12].y < lm[10].y # Jari tengah
            ring_up = lm[16].y < lm[14].y # Jari manis
            pinky_up = lm[20].y < lm[18].y # Jari kelingking

            fingers_up_count = sum([index_up, middle_up, ring_up, pinky_up]) # Menghitung jumlah jari yang terangkat

            # DRAW = hanya jari telunjuk terangkat
            if index_up and not middle_up and not ring_up and not pinky_up:
                draw_counter += 1
                erase_counter = 0
                fist_counter = 0

            # ERASE = semua jari terbuka (terangkat)
            elif fingers_up_count == 4:
                erase_counter += 1
                draw_counter = 0
                fist_counter = 0

            # FIST = semua jari tertutup (tidak terangkat)
            elif fingers_up_count == 0:
                fist_counter += 1
                draw_counter = 0
                erase_counter = 0

            else:
                draw_counter = erase_counter = fist_counter = 0 # Reset konter jika tidak ada gestur yang jelas

            # Konfirmasi gestur setelah melewati GESTURE_CONFIRM_FRAMES
            if draw_counter >= GESTURE_CONFIRM_FRAMES:
                gesture_state = "DRAW"
            elif erase_counter >= GESTURE_CONFIRM_FRAMES:
                gesture_state = "ERASE"
            elif fist_counter >= GESTURE_CONFIRM_FRAMES:
                gesture_state = "FIST"
            else:
                gesture_state = "NONE"

            mode = gesture_state # Update mode tampilan

            # ================= SMOOTHING =================
            index_point = px(lm[8]) # Dapatkan koordinat ujung jari telunjuk

            if smoothed_point is None:
                smoothed_point = index_point
            else:
                # Terapkan smoothing pada titik jari telunjuk
                smoothed_point = (
                    int(SMOOTHING_ALPHA * index_point[0] + (1 - SMOOTHING_ALPHA) * smoothed_point[0]),
                    int(SMOOTHING_ALPHA * index_point[1] + (1 - SMOOTHING_ALPHA) * smoothed_point[1])
                )

            # ================= DRAW =================
            if gesture_state == "DRAW":

                if prev_point is None:
                    prev_point = smoothed_point # Inisialisasi prev_point jika baru mulai menggambar

                cv2.line(canvas, # Menggambar garis pada kanvas
                         prev_point,
                         smoothed_point,
                         current_color,
                         BRUSH_SIZE,
                         cv2.LINE_AA)

                prev_point = smoothed_point # Update prev_point untuk frame berikutnya
            else:
                prev_point = None # Reset prev_point jika tidak dalam mode DRAW

            # ================= ERASE =================
            if gesture_state == "ERASE":
                canvas = np.zeros_like(frame) # Mengatur ulang kanvas menjadi hitam (menghapus semua gambar)

            # ================= COLOR PICKER =================
```

```
            if gesture_state == "FIST":
                # Cek apakah jari telunjuk (smoothed_point) berada di atas salah satu kotak warna
                for i, color in enumerate(color_palette):
                    x1 = w - 60
                    y1 = 50 + i * 70
                    x2 = w - 10
                    y2 = y1 + 50

                    if x1 < smoothed_point[0] < x2 and y1 < smoothed_point[1] < y2:
                        current_color = color # Ganti warna kuas ke warna yang dipilih

        # ================ DRAW COLOR PANEL ================
        # Menggambar palet warna di sisi kanan frame
        for i, color in enumerate(color_palette):
            x1 = w - 60
            y1 = 50 + i * 70
            x2 = w - 10
            y2 = y1 + 50

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, -1) # Mengisi kotak warna
            cv2.rectangle(frame, (x1, y1), (x2, y2), (255,255,255), 2) # Menggambar batas putih

        # ================ DISPLAY ================
        # Menggabungkan frame kamera dan kanvas gambar
        combined = cv2.addWeighted(frame, 0.8, canvas, 1, 0)

        # Menampilkan mode gestur saat ini
        cv2.putText(combined,
                    f"MODE: {mode}",
                    (20, 50),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    1,
                    (255, 255, 255),
                    2)

        cv2.imshow("AR Writing - Real Feel", combined) # Menampilkan hasil akhir

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break # Keluar dari loop jika tombol 'q' ditekan

    # ================ CLEANUP ================
    cap.release() # Melepaskan kamera
    cv2.destroyAllWindows() # Menutup semua jendela OpenCV
```

Penjelasan Rinci Loop Utama:

1. **Pembacaan Frame**: `cap.read()` membaca satu frame dari kamera.
2. **Flip Frame**: `cv2.flip(frame, 1)` membalik frame secara horizontal agar terlihat seperti cermin, yang lebih intuitif bagi pengguna.
3. **Inisialisasi Kanvas**: Jika `canvas` belum ada (saat pertama kali loop berjalan), ia diinisialisasi sebagai array NumPy kosong (hitam) dengan dimensi yang sama dengan frame kamera.
4. **Konversi Warna untuk MediaPipe**: Frame dikonversi dari BGR (format default OpenCV) ke RGB karena MediaPipe biasanya mengharapkan input RGB. Kemudian dibuat objek `mp.Image`.
5. **Deteksi Landmark Tangan**: `landmarker.detect_for_video()` menjalankan model MediaPipe untuk mendeteksi landmark tangan pada `mp_image` dengan `timestamp` yang diberikan.
6. **Penentuan Status Jari**: Jika tangan terdeteksi (`result.hand_landmarks` tidak kosong):
   - Koordinat landmark tangan pertama (`lm`) diambil.
   - Fungsi `px(p)` mengonversi koordinat relatif (0.0-1.0) dari landmark menjadi koordinat piksel absolut pada frame.
   - Setiap jari (telunjuk, tengah, manis, kelingking) diperiksa apakah terangkat (ujungnya lebih tinggi secara Y daripada pangkalnya).
   - `fingers_up_count` menghitung berapa banyak jari yang terangkat.
7. **Pengenalan Gestur**: Berdasarkan `fingers_up_count`:
   - **DRAW**: Hanya jari telunjuk yang terangkat. Konter `draw_counter` bertambah.
   - **ERASE**: Semua empat jari terangkat. Konter `erase_counter` bertambah.
   - **FIST**: Semua jari tertutup (tidak ada yang terangkat). Konter `fist_counter` bertambah.
   - Jika tidak ada gestur yang jelas, semua konter direset.
   - Setelah konter gestur mencapai `GESTURE_CONFIRM_FRAMES`, `gesture_state` diperbarui (misalnya, menjadi "DRAW"). Ini mencegah pemicuan gestur yang tidak disengaja.
8. **Smoothing Titik Jari**: Koordinat ujung jari telunjuk (`lm[8]`) diambil (`index_point`). `smoothed_point` dihitung dengan menggabungkan `index_point` saat ini dan `smoothed_point` sebelumnya menggunakan `SMOOTHING_ALPHA`. Ini membuat pergerakan kuas lebih halus dan mengurangi jitter.
9. **Aksi Menggambar**: Jika `gesture_state` adalah "DRAW":
   - `prev_point` diinisialisasi jika ini adalah awal gestur menggambar.
   - `cv2.line()` menggambar garis dari `prev_point` ke `smoothed_point` pada `canvas` dengan `current_color` dan `BRUSH_SIZE`.

    - `prev_point` diperbarui ke `smoothed_point` untuk garis berikutnya.
10. **Aksi Menghapus**: Jika `gesture_state` adalah "ERASE", `canvas` direset menjadi array hitam kosong, secara efektif menghapus semua gambar.
11. **Pemilihan Warna**: Jika `gesture_state` adalah "FIST":
    - Loop melalui `color_palette` untuk menggambar kotak-kotak warna di sisi kanan layar.
    - Memeriksa apakah `smoothed_point` (posisi jari telunjuk) berada di dalam salah satu kotak warna. Jika ya, `current_color` diperbarui ke warna yang dipilih.
12. **Menggambar Palet Warna**: Palet warna digambar di sisi kanan frame kamera. Kotak diisi dengan warna masing-masing dan diberi batas putih.
13. **Penggabungan Tampilan**: `cv2.addWeighted()` menggabungkan `frame` kamera (dengan opasitas 0.8) dan `canvas` (dengan opasitas 1). Ini menciptakan efek di mana gambar tampak transparan di atas umpan kamera.
14. **Teks Mode**: Menampilkan teks "MODE: [current_mode]" di sudut kiri atas layar.
15. **Tampilan Akhir**: `cv2.imshow()` menampilkan frame gabungan ke jendela bernama "AR Writing - Real Feel".
16. **Keluar**: Loop terus berjalan hingga tombol 'q' ditekan, yang memicu `break` dari loop.

5. Pembersihan

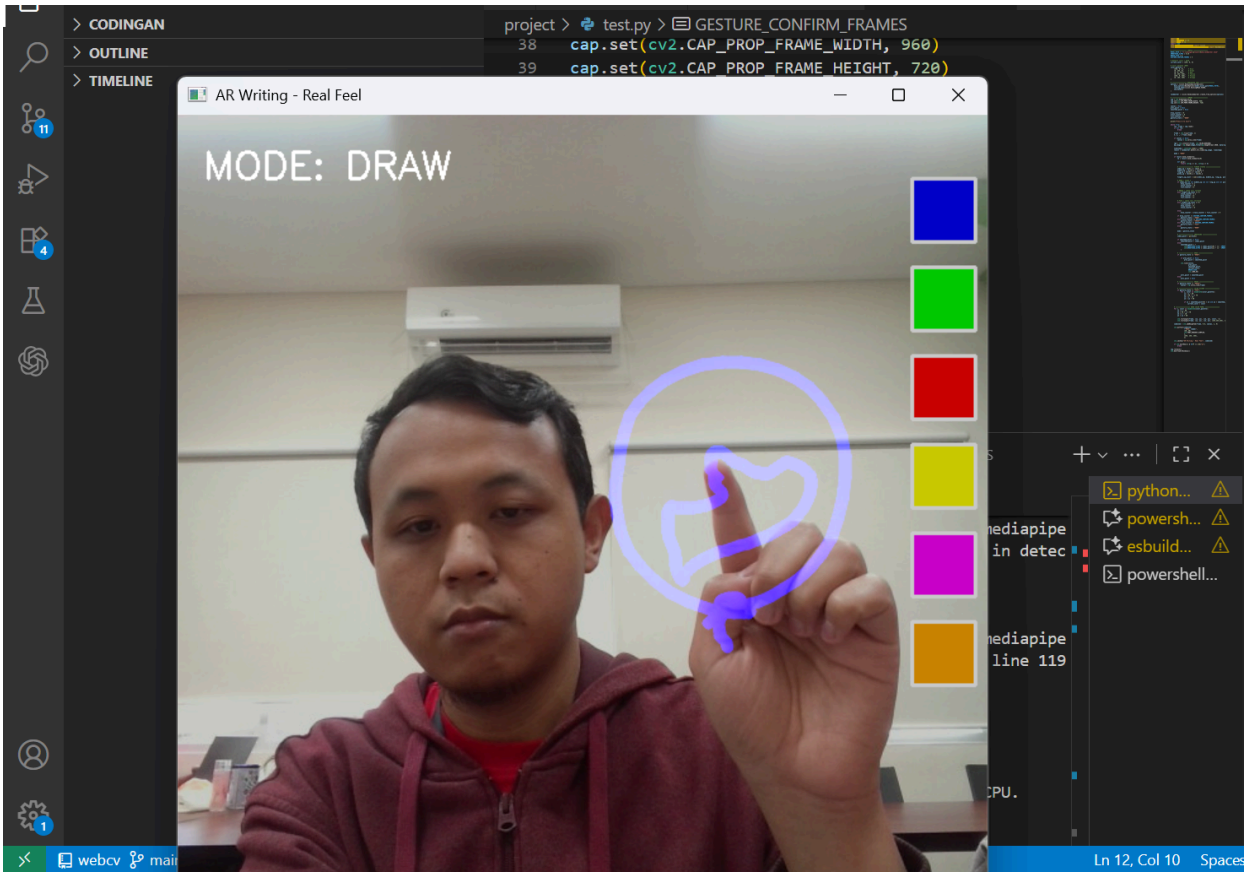
Setelah loop utama selesai, sumber daya kamera dilepaskan dan semua jendela OpenCV ditutup.

```
# ================= CLEANUP =================
cap.release() # Melepaskan kamera
cv2.destroyAllWindows() # Menutup semua jendela OpenCV
```

Penjelasan:

- `cap.release()`: Melepaskan objek `VideoCapture`, membebaskan sumber daya kamera.
- `cv2.destroyAllWindows()`: Menutup semua jendela yang dibuat oleh OpenCV.

Ringkasan Gestur:

- **Menggambar**: Angkat hanya jari telunjuk.
- **Menghapus**: Angkat semua jari (tangan terbuka).
- **Memilih Warna**: Kepalkan tangan dan arahkan jari telunjuk ke kotak warna di palet.

Contoh Hasil